

Homework 3 — Supervised Learning

Statistical and Mathematical Methods for AI

Indice

1	Obiettivo dell'homework	2
2	Esercizio 1: Regressione logistica su un dataset 2D giocattolo	3
3	Esercizio 2: SGD sulla regressione logistica	9
4	Esercizio 3: Metriche di valutazione	14
5	Esercizio 4: Regressione logistica su un dataset reale	19
6	Estensione opzionale: da regressione logistica a una rete neurale	27

1 Obiettivo dell'homework

Questo homework applica la Gradient Descent e la SGD (HW1, HW2) al problema della **classificazione binaria** tramite regressione logistica, ed estende l'analisi in tre direzioni:

- **Esercizio 1** implementa la regressione logistica da zero su un dataset 2D giocattolo, derivando il gradiente della binary cross-entropy e verificando perché la frontiera di decisione è sempre lineare.
- **Esercizio 2** allena lo stesso modello con SGD invece che GD a batch pieno, collegando il comportamento osservato alla teoria della varianza del gradiente stocastico già sviluppata in HW2.
- **Esercizio 3** introduce le metriche di valutazione per la classificazione (matrice di confusione, precision, recall) e il ruolo della soglia di decisione.
- **Esercizio 4** applica tutto quanto sopra a un dataset reale (Pima Indians Diabetes), confrontando SGD e **Adam** come ottimizzatori, e un'estensione opzionale introduce una piccola rete neurale con uno strato nascosto.

Il filo conduttore è che la regressione logistica compone un modello lineare $\Theta^T x$ con la funzione sigmoide $\sigma(\cdot)$: questo produce un confine di decisione sempre lineare (indipendentemente dalla soglia scelta), rende il gradiente della BCE identico in forma a quello della loss quadratica di HW1/HW2 ($\propto (\text{predizione} - \text{etichetta}) \times \text{input}$), e permette di riusare senza modifiche l'intero arsenale di ottimizzatori già sviluppato (GD, SGD, e qui anche Adam).

2 Esercizio 1: Regressione logistica su un dataset 2D giocattolo

Consegna

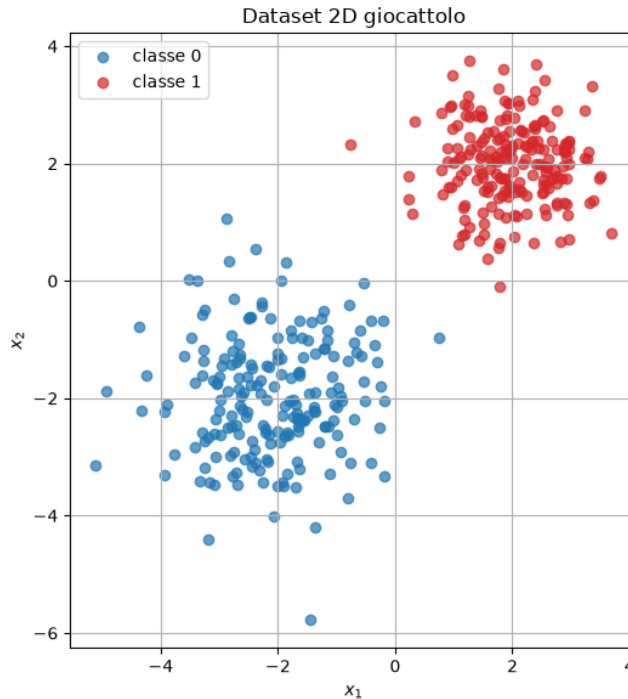
Generate two Gaussian clusters in $\mathbb{R}^2$: class 0 centered at $(-2, -2)$ with variance 1, class 1 centered at $(2, 2)$ with variance 0.5.

1. Plot the dataset in 2D, colored by class.
2. Implement logistic regression **from scratch**: $f_{\Theta}(x) = \sigma(\Theta^T x)$, $\ell(\Theta; x, y) = -[y \log f_{\Theta}(x) + (1 - y) \log(1 - f_{\Theta}(x))]$.
3. Train it using simple Gradient Descent on the full dataset.
4. Visualize the learned decision boundary: plot the line $\{\Theta^T x = 0\}$, overlaid with the dataset.
5. Comment on why the decision boundary is linear.

In [1]:

```
1  rng = np.random.default_rng(0)
2
3  N_per_class = 200
4  X0 = rng.normal(loc=[-2, -2], scale=np.sqrt(1.0), size=(N_per_class
5  , 2))
6  X1 = rng.normal(loc=[2, 2], scale=np.sqrt(0.5), size=(N_per_class
7  , 2))
8
9  y0 = np.zeros(N_per_class)
10 y1 = np.ones(N_per_class)
11
12
13 # shuffle
14 perm = rng.permutation(N)
15 X, y = X[perm], y[perm]
16
17 fig, ax = plt.subplots(figsize=(6, 6))
18 ax.scatter(X0[:,0], X0[:,1], color='tab:blue', label='classe 0',
19           alpha=0.7)
20 ax.scatter(X1[:,0], X1[:,1], color='tab:red', label='classe 1',
21           alpha=0.7)
22 ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
23 ax.set_title('Dataset 2D giocattolo')
24 ax.legend()
25 ax.set_aspect('equal')
26 plt.tight_layout()
27 plt.show()
```

Out[1]:



Due cluster gaussiani: classe 0 in $(-2, -2)$ (varianza 1), classe 1 in $(2, 2)$ (varianza 0.5). `np.random.normal` vuole la deviazione standard, non la varianza: da qui `scale=np.sqrt(varianza)` nel codice.

In [2]:

```

1  def sigmoid(z):
2      return 1.0 / (1.0 + np.exp(-z))
3
4  def bce_loss(theta, X_, y_):
5      z = X_ @ theta
6      p = sigmoid(z)
7      eps = 1e-12
8      return -np.mean(y_*np.log(p+eps) + (1-y_)*np.log(1-p+eps))
9
10 def bce_grad(theta, X_, y_):
11     p = sigmoid(X_ @ theta)
12     return (1.0/X_.shape[0]) * (X_.T @ (p - y_))
13
14 def accuracy(theta, X_, y_, threshold=0.5):
15     p = sigmoid(X_ @ theta)
16     pred = (p >= threshold).astype(float)
17     return np.mean(pred == y_)
18
19 X_tilde = np.column_stack([np.ones(N), X]) # colonna di bias
20
21 def gd_logreg(X_, y_, theta0, eta, n_iters):
22     theta = theta0.copy()
23     losses = [bce_loss(theta, X_, y_)]
24     for _ in range(n_iters):
25         theta = theta - eta * bce_grad(theta, X_, y_)
26         losses.append(bce_loss(theta, X_, y_))
27     return theta, np.array(losses)
28

```

```

29 theta0 = np.zeros(3)
30 theta_gd, losses_gd = gd_logreg(X_tilde, y, theta0, eta=0.5,
    n_iters=200)
31
32 print("theta appreso (bias, w1, w2):", theta_gd)
33 print(f"loss finale = {losses_gd[-1]:.6f}, accuracy finale = {
    accuracy(theta_gd, X_tilde, y):.4f}")

```

Out[2]:

```

1 theta appreso (bias, w1, w2): [-0.2533458  1.91538578  1.98092708]
2 loss finale = 0.005526, accuracy finale = 1.0000

```

$f_{\Theta}(x) = \sigma(\Theta^T \tilde{x})$ con $\tilde{x} = [1, x_1, x_2]$ (il bias è incluso esplicitamente, coerentemente con la convenzione usata in HW1/HW2). Il gradiente della BCE, $\nabla_{\Theta} \mathcal{L} = \frac{1}{N} X^T (f_{\Theta}(X) - Y)$, ha **la stessa forma** del gradiente MSE di HW1/HW2 — “residuo” $\times$ “input” — nonostante la non linearità della sigmoide: è una conseguenza generale dell’uso della BCE come loss associata a un output sigmoide.

In [3]:

```

1 s = X_tilde @ theta_gd
2 print(f"score minimo fra i punti di classe 1: {s[y==1].min():.3f}")
3 print(f"score massimo fra i punti di classe 0: {s[y==0].max():.3f}"
    )
4 print(f"-> i due gruppi sono separati da un iperpiano: {s[y==1].min
    () > s[y==0].max()}")
5 print()
6
7 print(f"{'iterazioni di GD':>18} {'||theta||':>11} {'loss':>12} {'
    accuracy':>10}")
8 for n_it in [200, 1000, 5000]:
9     th, ls = gd_logreg(X_tilde, y, theta0, eta=0.5, n_iters=n_it)
10    print(f"{n_it:>18} {np.linalg.norm(th):>11.3f} {ls[-1]:>12.6f}
    "
11          f"{accuracy(th, X_tilde, y):>10.4f}")
12
13 p_gd = sigmoid(X_tilde @ theta_gd)
14 print()
15 print(f"probabilita' predette: {np.sum((p_gd>0.3)&(p_gd<0.7))}
    punti su {N} cadono in [0.3, 0.7]")

```

Out[3]:

```

1 score minimo fra i punti di classe 1: 2.586
2 score massimo fra i punti di classe 0: -0.705
3 -> i due gruppi sono separati da un iperpiano: True
4
5   iterazioni di GD   ||theta||      loss    accuracy
6             200        2.767    0.005526    1.0000
7             1000        3.860    0.001844    1.0000
8             5000        5.312    0.000532    1.0000
9
10 probabilita' predette: 1 punti su 400 cadono in [0.3, 0.7]

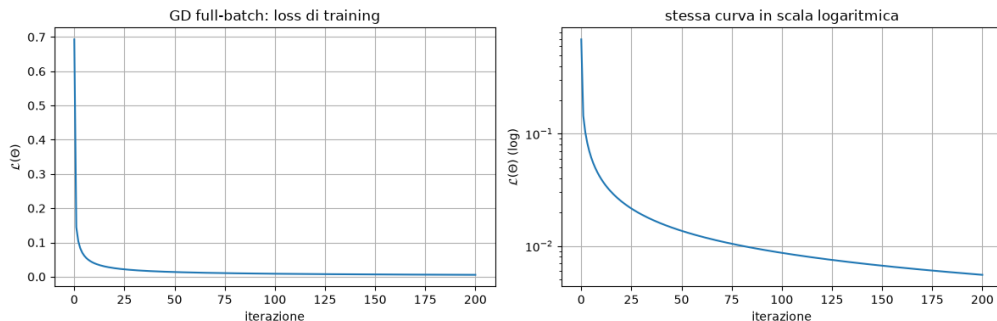
```

Un fatto teorico importante, verificato qui numericamente: i due cluster sono linearmente separabili. Se lo sono, la BCE non ha un minimo finito: qualunque iperpiano separatore può essere reso “più sicuro” moltiplicando Θ per una costante > 1 , facendo scendere la loss verso 0 e $\|\Theta\|$ verso infinito. La tabella lo conferma: aumentando le iterazioni da 200 a 5000, $\|\Theta\|$ cresce monotonicamente da 2.77 a 5.31 mentre la loss continua a scendere ($0.0055 \rightarrow 0.0005$) — non sta convergendo a un punto fisso, sta divergendo in norma pur avendo già accuracy 1.0 fin dall’inizio. In questo caso Θ appreso dipende da *quando ci si ferma*, non da un ottimo ben definito, e le probabilità predette diventano quasi tutte ≈ 0 o ≈ 1 (solo 1 punto su 400 cade in $[0.3, 0.7]$) — un dettaglio che condiziona direttamente la lettura degli Esercizi 2 e 3.

In [4]:

```
1 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
2 axes[0].plot(losses_gd)
3 axes[0].set_xlabel('iterazione'); axes[0].set_ylabel(r'\mathcal{L}(\Theta)')
4 axes[0].set_title('GD full-batch: loss di training')
5 axes[1].semilogy(losses_gd)
6 axes[1].set_xlabel('iterazione'); axes[1].set_ylabel(r'\mathcal{L}(\Theta) (log)')
7 axes[1].set_title('stessa curva in scala logaritmica')
8 plt.tight_layout()
9 plt.show()
```

Out[4]:



Loss di training in scala lineare (sinistra) e logaritmica (destra), 200 iterazioni di GD full-batch.

In [5]:

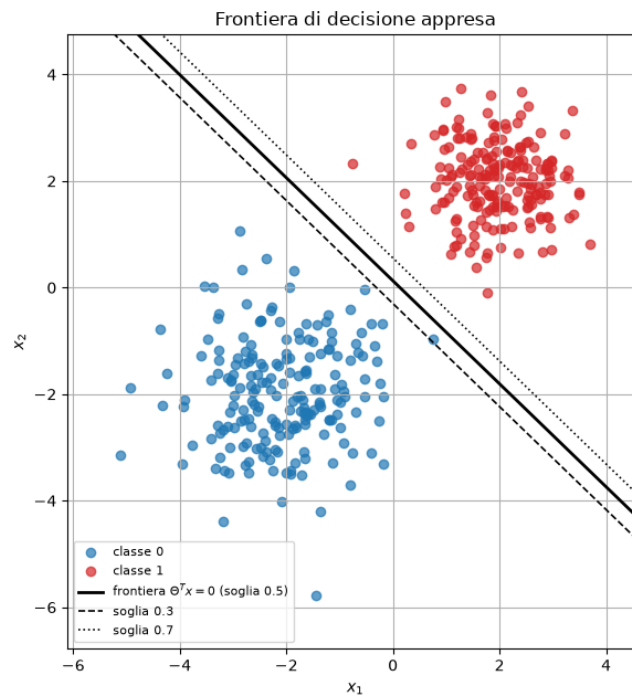
```
1 fig, ax = plt.subplots(figsize=(6.5, 6.5))
2 ax.scatter(X0[:,0], X0[:,1], color='tab:blue', label='classe 0',
3           alpha=0.7)
4 ax.scatter(X1[:,0], X1[:,1], color='tab:red', label='classe 1',
5           alpha=0.7)
6 # frontiera: theta0 + theta1*x1 + theta2*x2 = 0 => x2 = -(theta0 + theta1*x1)/theta2
7 x1_line = np.linspace(X[:,0].min()-1, X[:,0].max()+1, 100)
8 x2_line = -(theta_gd[0] + theta_gd[1]*x1_line) / theta_gd[2]
9 ax.plot(x1_line, x2_line, 'k-', lw=2, label=r'frontiera \Theta^Tx = 0$ (soglia 0.5)')
```

```

9
10 # rette a probabilita' costante 0.3 e 0.7:  $\Theta^T x = \log(t/(1-t))$ 
    )
11 for t, style in [(0.3, '--'), (0.7, ':')]:
12     c = np.log(t/(1-t))
13     ax.plot(x1_line, (c - theta_gd[0] - theta_gd[1]*x1_line)/
14             theta_gd[2], 'k'+style, lw=1.2,
15             label=f'soglia {t}')
16 ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
17 ax.set_title('Frontiera di decisione appresa')
18 ax.legend(fontsize=8)
19 ax.set_aspect('equal')
20 ax.set_xlim(X[:,0].min()-1, X[:,0].max()+1)
21 ax.set_ylim(X[:,1].min()-1, X[:,1].max()+1)
22 plt.tight_layout()
23 plt.show()

```

Out [5]:



Frontiera $\{\Theta^T x = 0\}$ (soglia 0.5, linea continua) insieme alle frontiere a soglia 0.3 e 0.7 (tratteggiate). Le tre rette sono quasi sovrapposte: coerente con il fatto che quasi tutti i punti hanno probabilità predetta vicinissima a 0 o 1 (Esercizio 1, punto separabilità).

Perché la frontiera di decisione è lineare. La frontiera è l'insieme $\{x : f_{\Theta}(x) = 0.5\}$. Poiché σ è **monotona** e $\sigma(0) = 0.5$:

$$f_{\Theta}(x) = 0.5 \iff \sigma(\Theta^T x) = 0.5 \iff \Theta^T x = 0.$$

$\Theta^T x = 0$ è un **iperpiano** (una retta in 2D) — la sigmoide non cambia la *forma* della frontiera, cambia solo quanto “sicuro” è il modello lontano da essa. È per questo che le tre rette nell’immagine (soglie 0.3, 0.5, 0.7) sono tutte **parallele tra loro**: cambiare la soglia t sposta semplicemente la retta di un’offset costante $c = \log(t/(1-t))/\|\cdot\|$ lungo la stessa

direzione normale Θ , senza mai cambiarne l'orientamento. Questo spiega anche perché la regressione logistica può separare bene solo classi (approssimativamente) linearmente separabili — come verificato essere proprio il caso qui.

3 Esercizio 2: SGD sulla regressione logistica

Consegna

Use the same synthetic dataset used in the previous exercise, but train logistic regression using **SGD**.

1. Try different batch sizes: $N_{\text{batch}} = 1, 10, N$ (full GD).
2. For each setting: plot the loss vs epoch, plot the classification accuracy vs epoch.
3. Compare the stability and speed of convergence over the choice of different batch sizes.
4. Why the gradients become noisier for small batches? Why larger batches give smoother curves?

In [1]:

```
1 def sgd_logreg(X_, y_, theta0, eta, batch_size, n_epochs, seed=0):
2     rng_ = np.random.default_rng(seed)
3     theta = theta0.copy()
4     N_ = X_.shape[0]
5     epoch_losses = [bce_loss(theta, X_, y_)]
6     epoch_accs = [accuracy(theta, X_, y_)]
7     iter_losses = [bce_loss(theta, X_, y_)]           # loss sull'intero
                                                dataset dopo ogni update
8     for epoch in range(n_epochs):
9         idx = rng_.permutation(N_)
10        for start in range(0, N_, batch_size):
11            b_idx = idx[start:start+batch_size]
12            g = bce_grad(theta, X_[b_idx], y_[b_idx])
13            theta = theta - eta * g
14            iter_losses.append(bce_loss(theta, X_, y_))
15            epoch_losses.append(bce_loss(theta, X_, y_))
16            epoch_accs.append(accuracy(theta, X_, y_))
17        return theta, np.array(epoch_losses), np.array(epoch_accs), np.
            array(iter_losses)
18
19 batch_sizes2 = [1, 10, N]
20 eta2 = 0.5
21 n_epochs2 = 60
22
23 results2 = {}
24 print(f"{'batch':>6} {'update/epoca':>13} {'loss finale':>12} {'
25         accuracy finale':>16} "
26       f"{'epoca in cui acc=1':>20}")
27 for bs in batch_sizes2:
28     theta_f, losses, accs, iter_losses = sgd_logreg(X_tilde, y, np.
29         zeros(3), eta2, bs,
30                                                     n_epochs2, seed
31                                                     =1)
32     results2[bs] = {'theta': theta_f, 'losses': losses, 'accs':
33         accs, 'iter_losses': iter_losses}
34     first = np.argmax(accs >= 1.0) if np.any(accs >= 1.0) else -1
35     print(f"{bs:>6} {int(np.ceil(N/bs)):>13} {losses[-1]:>12.5f} {
36         accs[-1]:>16.4f} "
37           f"{'first':>20}")
```

Out[1]:

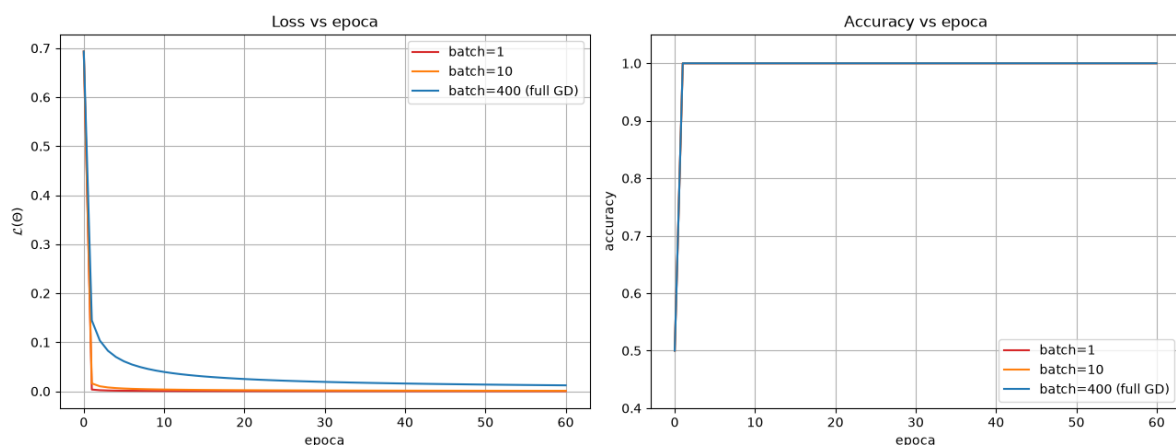
	batch	update/epoca	loss finale	accuracy finale	epoca in cui acc=1
1	1	400	0.00014	1.0000	1
2	10	40	0.00095	1.0000	1
3	400	1	0.01210	1.0000	1

Primo campanello d'allarme. Tutti e tre i metodi raggiungono $\text{accuracy}=1.0$ già alla **prima** epoca — coerente col fatto, verificato nell'Esercizio 1, che questo dataset è linearmente separabile e con cluster molto distanti tra loro. Questo significa che la curva di accuracy da sola **non distinguerà i tre metodi**: serve guardare la loss (che continua a scendere anche dopo aver raggiunto accuracy perfetta, per lo stesso motivo di $\|\Theta\| \rightarrow \infty$ discusso in Esercizio 1) e, per vedere davvero le differenze di stabilità, un dataset con classi più sovrapposte (celle successive).

In [2]:

```
1 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
2 colors2 = {1: 'tab:red', 10: 'tab:orange', N: 'tab:blue'}
3
4 ax = axes[0]
5 for bs in batch_sizes2:
6     label = f'batch={bs}' + (' (full GD)' if bs == N else '')
7     ax.plot(results2[bs]['losses'], color=colors2[bs], label=label)
8 ax.set_xlabel('epoca'); ax.set_ylabel(r'$\mathcal{L}(\Theta)$')
9 ax.set_title('Loss vs epoca')
10 ax.legend()
11
12 ax2 = axes[1]
13 for bs in batch_sizes2:
14     label = f'batch={bs}' + (' (full GD)' if bs == N else '')
15     ax2.plot(results2[bs]['accs'], color=colors2[bs], label=label)
16 ax2.set_xlabel('epoca'); ax2.set_ylabel('accuracy')
17 ax2.set_ylim(0.4, 1.05)
18 ax2.set_title('Accuracy vs epoca')
19 ax2.legend()
20
21 plt.tight_layout()
22 plt.show()
```

Out[2]:



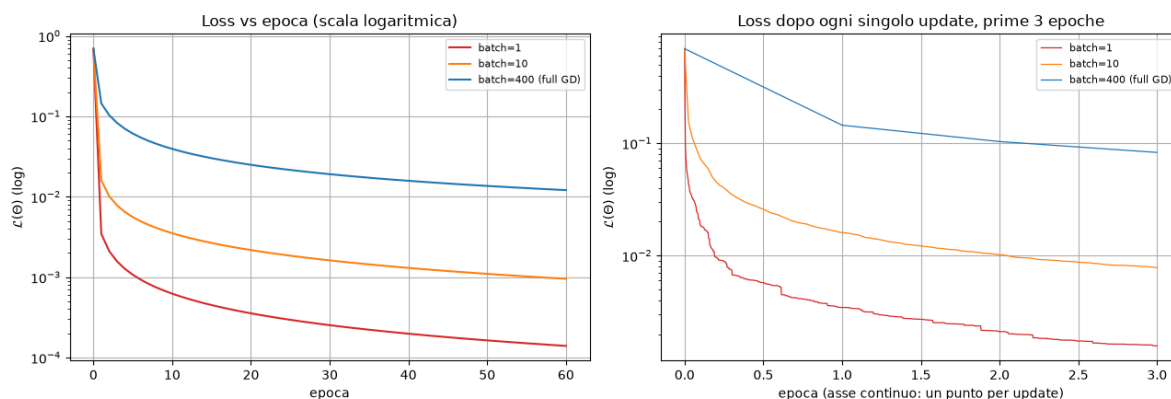
Loss vs epoca (sinistra, scala lineare --- si notino i valori estremamente piccoli sull'asse y) e accuracy vs epoca (destra), per batch $\in \{1, 10, 400\}$.

Confermato: nel pannello destro le tre curve di accuracy sono **indistinguibili**, tutte saturate a 1.0 dalla prima epoca. Nel pannello sinistro, in scala lineare, le tre curve di loss sembrano tutte schiacciate vicino a zero — serve la scala logaritmica (celle successive) per vederle davvero.

In [3]:

```
1  fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
2
3  ax = axes[0]
4  for bs in batch_sizes2:
5      ax.semilogy(results2[bs]['losses'], color=colors2[bs],
6                  label=f'batch={bs}' + (' (full GD)' if bs == N else
7                  ''))
8  ax.set_xlabel('epoca'); ax.set_ylabel(r'$\mathcal{L}(\Theta)$ (log)')
9  ax.set_title('Loss vs epoca (scala logaritmica)')
10 ax.legend(fontsize=8)
11
12 ax2 = axes[1]
13 for bs in batch_sizes2:
14     n_upd = int(np.ceil(N/bs)) * 3           # primi 3 epoche di
15     update                                     update
16     il = results2[bs]['iter_losses'][:n_upd+1]
17     ax2.semilogy(np.linspace(0, 3, len(il)), il, color=colors2[bs],
18                 lw=0.9,
19                 label=f'batch={bs}' + (' (full GD)' if bs == N
20                 else ''))
21 ax2.set_xlabel('epoca (asse continuo: un punto per update)')
22 ax2.set_ylabel(r'$\mathcal{L}(\Theta)$ (log)')
23 ax2.set_title('Loss dopo ogni singolo update, prime 3 epoche')
24 ax2.legend(fontsize=8)
25
26 plt.tight_layout()
27 plt.show()
```

Out [3]:



Sinistra: stessa loss di prima, ora in scala log --- qui le tre curve finalmente si separano. Destra: loss dopo ogni singolo update (non solo a fine epoca) nelle prime 3 epoche, asse x continuo.

Un risultato controintuitivo, visibile solo in scala log. Ci si aspetterebbe che batch= 1 (il più rumoroso) sia anche il *peggiore*, ma qui è l'opposto: dopo 60 epoche, batch= 1 raggiunge la loss **più bassa** di tutte (1.4×10^{-4}), seguito da batch= 10 (9×10^{-4}), con il full GD ultimo (1.2×10^{-2}). La spiegazione sta nel numero di update per epoca: batch= 1 ne fa 400 per epoca contro l'1 del full GD — essendo i dati linearmente separabili (Esercizio 1), più update significano più opportunità di far crescere $\|\Theta\|$ e quindi ridurre ulteriormente la loss (che, ricordiamo, non ha un minimo finito qui). Il pannello di destra lo conferma in modo diretto: guardando i primi update (non le epoche intere), batch= 1 scende più rapidamente *per unità di dati processati*, semplicemente perché fa più aggiornamenti nello stesso intervallo.

Questo è un caso in cui il dataset scelto dalla consegna, essendo troppo facilmente separabile, non permette di osservare il comportamento “da manuale” (SGD rumorosa che converge a una regione, non a un punto) — da qui la necessità dell'esperimento supplementare con dati sovrapposti.

In [4]:

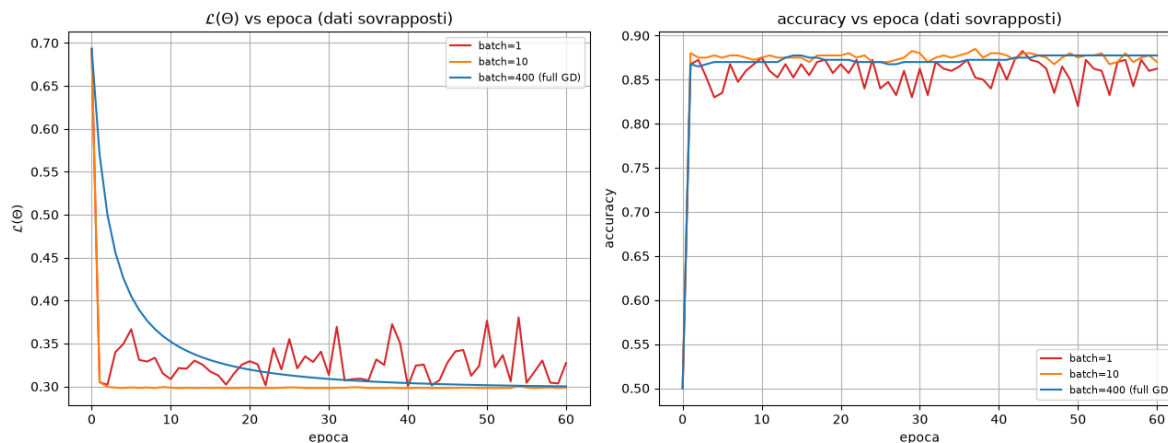
```

1  def make_overlapping(sep=0.75, n_per_class=200, seed=7):
2      r = np.random.default_rng(seed)
3      A = r.normal([-sep, -sep], 1.0, size=(n_per_class, 2))
4      B = r.normal([ sep,  sep], 1.0, size=(n_per_class, 2))
5      Xo = np.vstack([A, B])
6      yo = np.concatenate([np.zeros(n_per_class), np.ones(n_per_class
7                          )])
8      p = r.permutation(2*n_per_class)
9      return np.column_stack([np.ones(2*n_per_class), Xo[p]]), yo[p]
10
11 X_ov, y_ov = make_overlapping()
12 N_ov = X_ov.shape[0]
13
14 results2_ov = {}
15 print(f"{'batch':>6} {'loss finale':>12} {'accuracy finale':>16}")
16 for bs in [1, 10, N_ov]:
17     th, ls, ac, il = sgd_logreg(X_ov, y_ov, np.zeros(3), eta2, bs,
18                               n_epochs2, seed=1)
19     results2_ov[bs] = {'theta': th, 'losses': ls, 'accs': ac, '
20                       iter_losses': il}
21     print(f"{bs:>6} {ls[-1]:>12.5f} {ac[-1]:>16.4f}")
22
23 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
24 colors2_ov = {1: 'tab:red', 10: 'tab:orange', N_ov: 'tab:blue'}
25 for ax, key, name in [(axes[0], 'losses', r'$\mathcal{L}(\Theta)$'),
26                       (axes[1], 'accs', 'accuracy')]:
27     for bs in [1, 10, N_ov]:
28         ax.plot(results2_ov[bs][key], color=colors2_ov[bs],
29               label=f'batch={bs}' + (' (full GD)' if bs == N_ov
30                                   else ''))
31     ax.set_xlabel('epoca'); ax.set_ylabel(name)
32     ax.set_title(name + ' vs epoca (dati sovrapposti)')
33     ax.legend(fontsize=8)
34 plt.tight_layout()
35 plt.show()

```

Out [4]:

	batch	loss finale	accuracy finale
1	1	0.32725	0.8625
3	10	0.29871	0.8700
4	400	0.30009	0.8775



Stesso confronto, ma su un dataset con classi sovrapposte (centri a distanza 0.75 invece di 2-2.83, stessa varianza 1). Qui il comportamento classico emerge con chiarezza.

Qui finalmente si vede tutto quello che la consegna chiede di osservare. Con classi sovrapposte (accuracy massima raggiungibile < 1 , qui ≈ 0.86 - 0.88 , perché una frazione dei punti dei due cluster si mescola inevitabilmente), la loss ha un minimo finito ben definito e le differenze tra batch size emergono con chiarezza:

- **Stabilità e velocità di convergenza:** la curva blu (full GD) è perfettamente liscia e monotona ma parte più lentamente; la curva arancione (batch= 10) converge quasi altrettanto velocemente ma con una leggerissima rugosità; la curva rossa (batch= 1) è nettamente la più rumorosa, con oscillazioni visibili sia nella loss sia nell'accuracy per tutte le 60 epoche, senza mai stabilizzarsi su una linea piatta.
- **Perché i gradienti sono più rumorosi con batch piccoli:** con batch= 1, ogni aggiornamento usa il gradiente calcolato su un *singolo* punto — una stima ad alta varianza del gradiente vero (calcolato su tutti i 400 punti), che può puntare in una direzione anche molto diversa da esso.
- **Perché batch grandi danno curve più regolari:** mediando su più campioni, il gradiente di mini-batch si avvicina sempre di più al gradiente vero (varianza $\propto 1/N_{\text{batch}}$, come quantificato in HW2 Esercizio 2) — coerente con l'ordine di rugosità osservato: rosso (batch 1) > arancione (batch 10) > blu (full batch, nessun rumore).

4 Esercizio 3: Metriche di valutazione

Consegna

Using the logistic regression model trained above (Esercizio 1):

1. Compute predicted probabilities $\hat{y}_i = f_{\Theta}(x_i)$.
2. Convert them to binary predictions using threshold 0.5.
3. Compute: confusion matrix (TP, FP, FN, TN), accuracy, precision, recall, F1-score.
4. Modify the threshold to 0.3 and 0.7, and repeat.
5. Comment on: how lower thresholds increase recall and lower precision, how higher thresholds increase precision and reduce recall, why classification metrics depend on the application.

In [1]:

```
1 def confusion_matrix_counts(y_true, y_pred):
2     TP = np.sum((y_true==1) & (y_pred==1))
3     FP = np.sum((y_true==0) & (y_pred==1))
4     FN = np.sum((y_true==1) & (y_pred==0))
5     TN = np.sum((y_true==0) & (y_pred==0))
6     return TP, FP, FN, TN
7
8 def compute_metrics(theta, X_, y_, threshold=0.5):
9     p = sigmoid(X_ @ theta)
10    pred = (p >= threshold).astype(float)
11    TP, FP, FN, TN = confusion_matrix_counts(y_, pred)
12    acc = (TP+TN) / (TP+FP+FN+TN)
13    precision = TP / (TP+FP) if (TP+FP) > 0 else 0.0
14    recall = TP / (TP+FN) if (TP+FN) > 0 else 0.0
15    f1 = 2*precision*recall/(precision+recall) if (precision+recall
16    ) > 0 else 0.0
17    return {'TP': TP, 'FP': FP, 'FN': FN, 'TN': TN,
18            'accuracy': acc, 'precision': precision, 'recall':
19            recall, 'f1': f1}
20
21 metrics_05 = compute_metrics(theta_gd, X_tilde, y, threshold=0.5)
22 print("Soglia = 0.5")
23 for k, v in metrics_05.items():
24     print(f"    {k}: {v:.4f}" if isinstance(v, float) else f"    {k}: {
25         v}")
```

Out[1]:

```
1 Soglia = 0.5
2 TP: 200
3 FP: 0
4 FN: 0
5 TN: 200
6 accuracy: 1.0000
7 precision: 1.0000
8 recall: 1.0000
9 f1: 1.0000
```

In [2]:

```
1 thresholds = [0.3, 0.5, 0.7]
2 metrics_by_threshold = {t: compute_metrics(theta_gd, X_tilde, y,
    threshold=t) for t in thresholds}
3
4 print(f"{'soglia':>10s} {'TP':>4s} {'FP':>4s} {'FN':>4s} {'TN':>4s}
    {'accuracy':>9s} "
5       f"{'precision':>10s} {'recall':>8s} {'f1':>7s}")
6 for t in thresholds:
7     m = metrics_by_threshold[t]
8     print(f"{t:<10.1f} {m['TP']:>4d} {m['FP']:>4d} {m['FN']:>4d} {m
    ['TN']:>4d} "
9           f"{m['accuracy']:>9.4f} {m['precision']:>10.4f} {m['
    recall']:>8.4f} {m['f1']:>7.4f}")
```

Out[2]:

soglia	TP	FP	FN	TN	accuracy	precision	recall	f1
0.3	200	1	0	199	0.9975	0.9950	1.0000	0.9975
0.5	200	0	0	200	1.0000	1.0000	1.0000	1.0000
0.7	200	0	0	200	1.0000	1.0000	1.0000	1.0000

Le tre righe sono quasi identiche — solo 1 falso positivo a soglia 0.3, nient'altro cambia. Prima di trarre conclusioni sul trade-off precision/recall bisogna capire *perché*: la soglia sposta la decisione solo per i punti la cui probabilità predetta cade *fra* le soglie confrontate. Se quasi nessun punto ha probabilità intermedia, cambiare soglia non ha quasi nulla su cui agire.

In [3]:

```
1 p_toy = sigmoid(X_tilde @ theta_gd)
2 print(f"punti con probabilita' predetta in [0.3, 0.7]: {np.sum((
    p_toy>0.3)&(p_toy<0.7))} su {N}")
3 print(f"punti con probabilita' predetta in [0.05, 0.95]: {np.sum((
    p_toy>0.05)&(p_toy<0.95))} su {N}")
4
5 grid = np.linspace(0.01, 0.99, 99)
6 sweep = {name: [compute_metrics(theta_gd, X_tilde, y, threshold=t)[
    name] for t in grid]
7           for name in ['accuracy', 'precision', 'recall', 'f1']}
8
9 fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
10 ax = axes[0]
11 ax.hist(p_toy[y==0], bins=30, range=(0,1), alpha=0.7, color='tab:
    blue', label='classe 0')
12 ax.hist(p_toy[y==1], bins=30, range=(0,1), alpha=0.7, color='tab:
    red', label='classe 1')
13 for t in thresholds:
14     ax.axvline(t, color='k', ls='--', lw=1)
15 ax.set_xlabel("probabilita' predetta"); ax.set_ylabel('conteggio')
16 ax.set_title("Distribuzione delle probabilita' predette")
17 ax.legend(fontsize=8)
18
19 ax2 = axes[1]
20 for name, c in [('accuracy', 'tab:blue'), ('precision', 'tab:orange')
    ,
```

```

21         ('recall', 'tab:green'), ('f1', 'tab:red')]):
22     ax2.plot(grid, sweep[name], color=c, label=name)
23     ax2.set_xlabel('soglia'); ax2.set_ylabel('valore della metrica')
24     ax2.set_ylim(-0.02, 1.05)
25     ax2.set_title('Metriche al variare della soglia (griglia fitta)')
26     ax2.legend(fontsize=8)
27     plt.tight_layout()
28     plt.show()

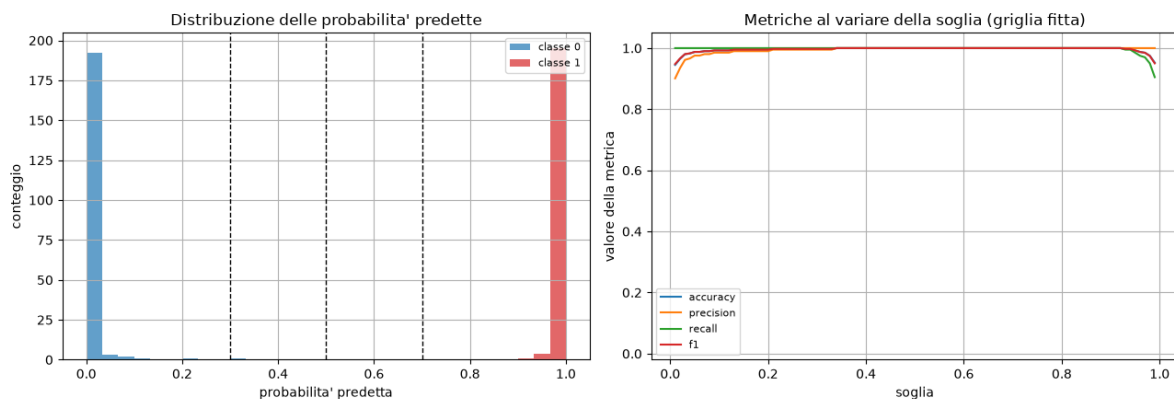
```

Out [3]:

```

1 punti con probabilita' predetta in [0.3, 0.7]: 1 su 400
2 punti con probabilita' predetta in [0.05, 0.95]: 8 su 400

```



Sinistra: istogramma delle probabilità predette, con le tre soglie tratteggiate. Destra: le quattro metriche su una griglia fitta di 99 soglie.

Confermato numericamente: solo 8 punti su 400 hanno probabilità predetta anche solo in $[0.05, 0.95]$ — il modello è quasi ovunque confidente al 100% (conseguenza diretta della separabilità lineare vista in Esercizio 1: $\|\Theta\|$ grande spinge $\sigma(\Theta^T x)$ verso 0 o 1 quasi ovunque). Il pannello di destra lo conferma: le quattro metriche restano piatte a 1.0 per quasi tutto l'intervallo $[0.05, 0.9]$, con un crollo visibile solo agli estremi. Il dataset della consegna, da solo, non permette di osservare un vero compromesso precision/recall — da qui l'esperimento supplementare.

In [4]:

```

1 theta_ov, _ = gd_logreg(X_ov, y_ov, np.zeros(3), eta=0.5, n_iters
    =2000)
2 p_ov = sigmoid(X_ov @ theta_ov)
3 print(f"punti con probabilita' predetta in [0.3, 0.7]: "
4       f"{np.sum((p_ov>0.3)&(p_ov<0.7))} su {N_ov}")
5 print()
6 print(f"{'soglia':>10s} {'TP':>4s} {'FP':>4s} {'FN':>4s} {'TN':>4s}
    {'accuracy':>9s} "
7       f"{'precision':>10s} {'recall':>8s} {'f1':>7s}")
8 for t in thresholds:
9     m = compute_metrics(theta_ov, X_ov, y_ov, threshold=t)
10    print(f"{'t':<10.1f} {m['TP']:>4d} {m['FP']:>4d} {m['FN']:>4d} {m
    ['TN']:>4d} "
11          f"{m['accuracy']:>9.4f} {m['precision']:>10.4f} {m['
    recall']:>8.4f} {m['f1']:>7.4f}")

```



```

12
13 sweep_ov = {name: [compute_metrics(theta_ov, X_ov, y_ov, threshold=
    t)[name] for t in grid]
14               for name in ['accuracy', 'precision', 'recall', 'f1']}
15
16 fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
17 ax = axes[0]
18 ax.hist(p_ov[y_ov==0], bins=30, range=(0,1), alpha=0.7, color='tab:
    blue', label='classe 0')
19 ax.hist(p_ov[y_ov==1], bins=30, range=(0,1), alpha=0.7, color='tab:
    red', label='classe 1')
20 for t in thresholds:
21     ax.axvline(t, color='k', ls='--', lw=1)
22 ax.set_xlabel("probabilita' predetta"); ax.set_ylabel('conteggio')
23 ax.set_title("Dati sovrapposti: probabilita' predette")
24 ax.legend(fontsize=8)
25
26 ax2 = axes[1]
27 for name, c in [('accuracy', 'tab:blue'), ('precision', 'tab:orange'),
    ,
28               ('recall', 'tab:green'), ('f1', 'tab:red')]:
29     ax2.plot(grid, sweep_ov[name], color=c, label=name)
30 ax2.set_xlabel('soglia'); ax2.set_ylabel('valore della metrica')
31 ax2.set_title('Dati sovrapposti: metriche vs soglia')
32 ax2.legend(fontsize=8)
33 plt.tight_layout()
34 plt.show()

```

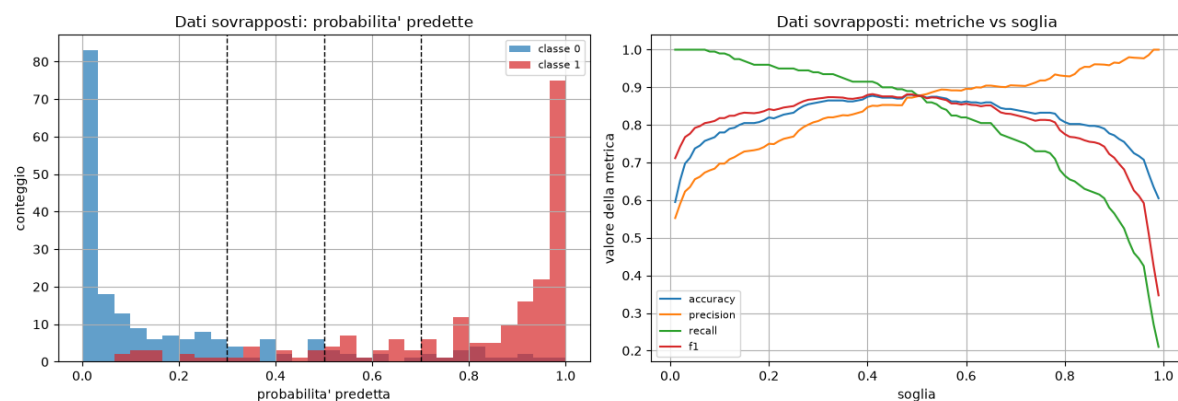
Out[4]:

```

1 punti con probabilita' predetta in [0.3, 0.7]: 64 su 400
2

```

soglia	TP	FP	FN	TN	accuracy	precision	recall	f1
0.3	188	44	12	156	0.8600	0.8103	0.9400	0.8704
0.5	176	25	24	175	0.8775	0.8756	0.8800	0.8778
0.7	152	16	48	184	0.8400	0.9048	0.7600	0.8261



Stesso esperimento sul dataset con classi sovrapposte: qui 64 punti su 400 hanno probabilità intermedia, abbastanza da mostrare il trade-off in modo netto.

Qui il trade-off precision/recall si vede in modo pulito, con i numeri a supporto.

- Soglie più basse aumentano il recall e abbassano la precision: da soglia 0.5

a 0.3, il recall sale da 0.880 a 0.940 (si perdono meno veri positivi: FN scende da 24 a 12), ma la precision scende da 0.876 a 0.810 (FP sale da 25 a 44) — più punti classificati positivi, quindi più veri positivi catturati, ma anche più falsi allarmi.

- **Soglie più alte aumentano la precision e riducono il recall:** da soglia 0.5 a 0.7, la precision sale da 0.876 a 0.905 (FP scende da 25 a 16: le poche predizioni positive sono più affidabili), ma il recall crolla da 0.880 a 0.760 (FN sale da 24 a 48: si perdono molti veri positivi con confidenza solo moderata).
- Il pannello destro della figura conferma l'andamento su tutta la griglia di soglie: precision (arancione) monotona crescente, recall (verde) monotona decrescente, che si incrociano attorno a soglia ≈ 0.45 — esattamente vicino al punto dove F1 (rosso) è massimo, coerente con la definizione di F1 come media armonica che bilancia le due.

Perché le metriche di classificazione dipendono dall'applicazione. Il costo relativo di un falso positivo vs un falso negativo dipende dal contesto: in uno screening medico, un falso negativo (malato non diagnosticato) è tipicamente molto più grave di un falso positivo (approfondimento inutile) $\Rightarrow$ si preferisce una soglia bassa, per massimizzare il recall anche a costo di precision — esattamente il caso del dataset reale trattato nell'Esercizio 4. In un filtro spam, un falso positivo (email importante marcata come spam) può essere peggiore di un falso negativo (spam non filtrato) $\Rightarrow$ si preferisce una soglia alta. Non esiste una soglia “giusta” in assoluto: la scelta va fatta in base ai costi reali dell'applicazione, non solo guardando l'accuratezza complessiva.

5 Esercizio 4: Regressione logistica su un dataset reale

Consegna

Reproduce the pipeline using the **Pima Indians Diabetes Dataset** (768 samples, medical measurements, binary diagnosis label).

1. Preprocess: download `diabetes.csv`, extract X, y , standardize features (mean 0, variance 1) — explain why, connecting to conditioning, stable optimization, meaningful gradient magnitudes — add bias column.
2. Implement logistic regression from scratch, optimize with **SGD** (batch 32, $\eta = 10^{-3}$, 200 epochs), tracking full-dataset BCE loss and accuracy per epoch. Plot Loss vs epoch and Accuracy vs epoch.
3. Train the same model with **Adam** (formule dalla teoria), stessi iperparametri. Plot SGD vs Adam.
4. For each method: final accuracy, confusion matrix, precision, recall, F1.
5. Discuss which method converges faster, which oscillates more, and how this relates to adaptive learning rates.

In [1]:

```
1 import pandas as pd
2
3 df = pd.read_csv('diabetes.csv')
4 print(df.shape)
5 df.head()
```

Out[1]:

```
1 (768, 9)
2      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI   \
3  0              6     148             72              35        0  33.6
4  1              1      85             66              29        0  26.6
5  ...
6      DiabetesPedigreeFunction  Age  Outcome
7  0                        0.627   50        1
8  1                        0.351   31        0
```

In [2]:

```
1 feature_cols4 = ['Pregnancies', 'Glucose', 'BloodPressure', '
2                 SkinThickness',
3                 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age
4                 'Outcome']
5
6 X4_raw = df[feature_cols4].values.astype(float)
7 y4 = df['Outcome'].values.astype(float)
8
9 X4_mean, X4_std = X4_raw.mean(axis=0), X4_raw.std(axis=0)
10 X4_scaled = (X4_raw - X4_mean) / X4_std
11 X4 = np.column_stack([np.ones(len(df)), X4_scaled])    # colonna di
12               bias
13
14 N4 = X4.shape[0]
```

```

11 d4 = X4.shape[1]
12 print(f"N = {N4}, numero di parametri (bias incluso) = {d4}")
13 print("Bilanciamento delle classi:", np.bincount(y4.astype(int)))
14 print()
15 print(f"{'feature':>26} {'media':>10} {'dev.std':>10} {'min':>8} {'max':>8}")
16 for j, c in enumerate(feature_cols4):
17     col = X4_raw[:, j]
18     print(f"{c:>26} {col.mean():>10.2f} {col.std():>10.2f} {col.min():>8.2f} {col.max():>8.2f}")

```

Out[2]:

```

1 N = 768, numero di parametri (bias incluso) = 9
2 Bilanciamento delle classi: [500 268]
3
4           feature      media    dev.std      min      max
5     Pregnancies      3.85      3.37      0.00     17.00
6         Glucose    120.89     31.95      0.00    199.00
7     BloodPressure    69.11     19.34      0.00    122.00
8     SkinThickness    20.54     15.94      0.00     99.00
9         Insulin     79.80    115.17      0.00    846.00
10             BMI     31.99      7.88      0.00     67.10
11 DiabetesPedigreeFunction 0.47      0.33      0.08      2.42
12             Age     33.24     11.75     21.00     81.00

```

Le feature vivono su scale numeriche molto diverse: Insulin arriva fino a 846, DiabetesPedigreeFunction resta sotto 2.5 — un rapporto di scala di oltre 300×. Dataset sbilanciato (500 vs 268), ma non estremamente ($\approx 65\%/35\%$): non richiede correzioni speciali per questo homework.

In [3]:

```

1 # Perché serve la standardizzazione: invece di argomentarlo solo a
  # parole, lo
2 # misuriamo. Per la regressione logistica l'Hessiana è  $\nabla^2 L =$ 
  #  $(1/N) X^T S X$ 
3 # con  $S = \text{diag}(f(x_i)(1-f(x_i)))$ : il condizionamento del problema è
  # quindi governato
4 # da quello di  $X^T X / N$ .
5 X4_unscaled = np.column_stack([np.ones(N4), X4_raw]) # stesse
  # feature, senza standardizzare
6
7 for nome, M in [('feature grezze', X4_unscaled), ('feature
  standardizzate', X4)]:
8     G = M.T @ M / N4
9     ev = np.linalg.eigvalsh(G)
10    theta_zero = np.zeros(M.shape[1])
11    g0 = bce_grad(theta_zero, M, y4)
12    print(f"{'nome':>24}: kappa( $X^T X / N$ ) = {ev[-1]/ev[0]:>12.3e}
      "
13          f"eta massimo ~ {2/ev[-1]:.2e}")
14    print(f"{'':>24} componenti del gradiente iniziale: min |g_j|
      = {np.abs(g0).min():.3e}, "
15          f"max |g_j| = {np.abs(g0).max():.3e}, rapporto = {np.abs(
      g0).max()/np.abs(g0).min():.1f}")

```

Out [3]:

```
1         feature grezze: kappa(X^T X / N) =      1.212e+06    eta massimo ~
                        5.81e-05
2                                componenti del gradiente iniziale: min |g_j| =
                        4.384e-02, max |g_j| = 1.115e+01, rapporto =
                        254.5
3     feature standardizzate: kappa(X^T X / N) =      5.178e+00    eta massimo ~
                        9.55e-01
4                                componenti del gradiente iniziale: min |g_j| =
                        3.101e-02, max |g_j| = 2.224e-01, rapporto =
                        7.2
```

Perché serve la standardizzazione, con i numeri a supporto. Per la regressione logistica $\nabla^2 \mathcal{L} = \frac{1}{N} X^T S X$ con S diagonale positiva: il condizionamento del problema è quindi governato da quello di $X^T X / N$, esattamente come per la loss quadratica di HW1/HW2.

- **Conditioning:** senza standardizzare, $\kappa \approx 1.2 \times 10^6$ — un condizionamento estremo, ordini di grandezza peggiore di qualsiasi esempio in HW1. Standardizzando, κ crolla a 5.18 — quasi ben condizionato quanto il caso $A = \text{diag}(5, 2)$ di HW1 Esercizio 4.
- **Ottimizzazione stabile:** la soglia teorica di stabilità $\eta < 2/\lambda_{\max}$ passa da 5.81×10^{-5} (feature grezze) a 0.955 (standardizzate) — un margine di stabilità $\sim 16000\times$ più ampio. Il $\eta = 10^{-3}$ usato nell'esercizio sarebbe stato $\sim 17\times$ sopra la soglia critica sui dati grezzi, garantendo divergenza; è invece ampiamente sicuro sui dati standardizzati.
- **Gradienti significativi:** sui dati grezzi, la componente più grande del gradiente iniziale è $254.5\times$ quella più piccola (dominata dalla feature *Insulin*, sulla scala più ampia) — un singolo η non può muovere in modo bilanciato tutti i parametri. Standardizzando, questo rapporto scende a 7.2: le componenti del gradiente sono molto più comparabili tra loro, e l'importanza relativa di ciascun parametro nell'ottimizzazione riflette la sua rilevanza reale, non la scala numerica arbitraria della feature corrispondente.

In [4]:

```
1 def sgd_logreg_tracked(X_, y_, theta0, eta, batch_size, n_epochs,
2 seed=0):
3     rng_ = np.random.default_rng(seed)
4     theta = theta0.copy()
5     N_ = X_.shape[0]
6     losses = [bce_loss(theta, X_, y_)]
7     accs = [accuracy(theta, X_, y_)]
8     thetas = [theta.copy()]
9     for epoch in range(n_epochs):
10         idx = rng_.permutation(N_)
11         for start in range(0, N_, batch_size):
12             b_idx = idx[start:start+batch_size]
13             g = bce_grad(theta, X_[b_idx], y_[b_idx])
14             theta = theta - eta * g
15             losses.append(bce_loss(theta, X_, y_))
16             accs.append(accuracy(theta, X_, y_))
17             thetas.append(theta.copy())
18     return theta, np.array(losses), np.array(accs), np.array(thetas)
```

```

19 eta_sgd = 1e-3
20 batch_size4 = 32
21 n_epochs4 = 200
22
23 theta_sgd, losses_sgd, accs_sgd, thetas_sgd = sgd_logreg_tracked(
24     X4, y4, np.zeros(d4), eta_sgd, batch_size4, n_epochs4, seed=4)
25 print(f"SGD: loss finale = {losses_sgd[-1]:.4f}, accuracy finale =
      {accs_sgd[-1]:.4f}, "
26       f"||theta|| = {np.linalg.norm(theta_sgd):.4f}")

```

Out[4]:

```

1 SGD: loss finale = 0.5088, accuracy finale = 0.7643, ||theta|| = 0.8697

```

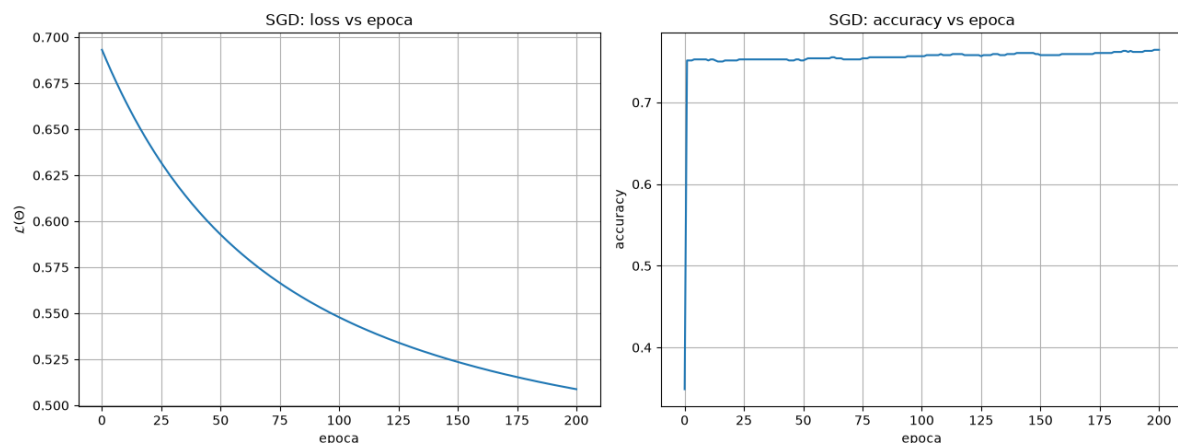
In [5]:

```

1 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
2 axes[0].plot(losses_sgd, color='tab:blue')
3 axes[0].set_xlabel('epoca'); axes[0].set_ylabel(r'$\mathcal{L}(\Theta)$')
4 axes[0].set_title('SGD: loss vs epoca')
5
6 axes[1].plot(accs_sgd, color='tab:blue')
7 axes[1].set_xlabel('epoca'); axes[1].set_ylabel('accuracy')
8 axes[1].set_title('SGD: accuracy vs epoca')
9
10 plt.tight_layout()
11 plt.show()

```

Out[5]:



SGD: loss (sinistra) e accuracy (destra) vs epoca, dataset reale.

La loss scende in modo fairly regolare ma è ancora in discesa attiva alla fine delle 200 epoche (non ha raggiunto un plateau netto): coerente con $\eta = 10^{-3}$, un passo cauto rispetto alla soglia teorica 0.955 appena calcolata. L'accuracy sale rapidamente nelle prime epoche e poi oscilla in una banda stretta attorno a 0.75-0.76.

In [6]:

```
1 def adam_logreg_tracked(X_, y_, theta0, eta, batch_size, n_epochs,
2                          beta1=0.9, beta2=0.999, epsilon=1e-8, seed
3                          =0):
4     rng_ = np.random.default_rng(seed)
5     theta = theta0.copy()
6     N_ = X_.shape[0]
7     m = np.zeros_like(theta)
8     v = np.zeros_like(theta)
9     t = 0
10    losses = [bce_loss(theta, X_, y_)]
11    accs = [accuracy(theta, X_, y_)]
12    thetas = [theta.copy()]
13    for epoch in range(n_epochs):
14        idx = rng_.permutation(N_)
15        for start in range(0, N_, batch_size):
16            b_idx = idx[start:start+batch_size]
17            g = bce_grad(theta, X_[b_idx], y_[b_idx])
18            t += 1
19            m = beta1*m + (1-beta1)*g
20            v = beta2*v + (1-beta2)*(g**2)
21            m_hat = m / (1 - beta1**t)
22            v_hat = v / (1 - beta2**t)
23            theta = theta - eta * m_hat / (np.sqrt(v_hat) + epsilon)
24            losses.append(bce_loss(theta, X_, y_))
25            accs.append(accuracy(theta, X_, y_))
26            thetas.append(theta.copy())
27    return theta, np.array(losses), np.array(accs), np.array(thetas)
28
29 theta_adam, losses_adam, accs_adam, thetas_adam =
30 adam_logreg_tracked(
31     X4, y4, np.zeros(d4), eta_sgd, batch_size4, n_epochs4, seed=4)
32 print(f"Adam: loss finale = {losses_adam[-1]:.4f}, accuracy finale
33       = {accs_adam[-1]:.4f}, "
34       f"||theta|| = {np.linalg.norm(theta_adam):.4f}")
```

Out[6]:

```
1 Adam: loss finale = 0.4710, accuracy finale = 0.7826, ||theta|| = 1.6895
```

In [7]:

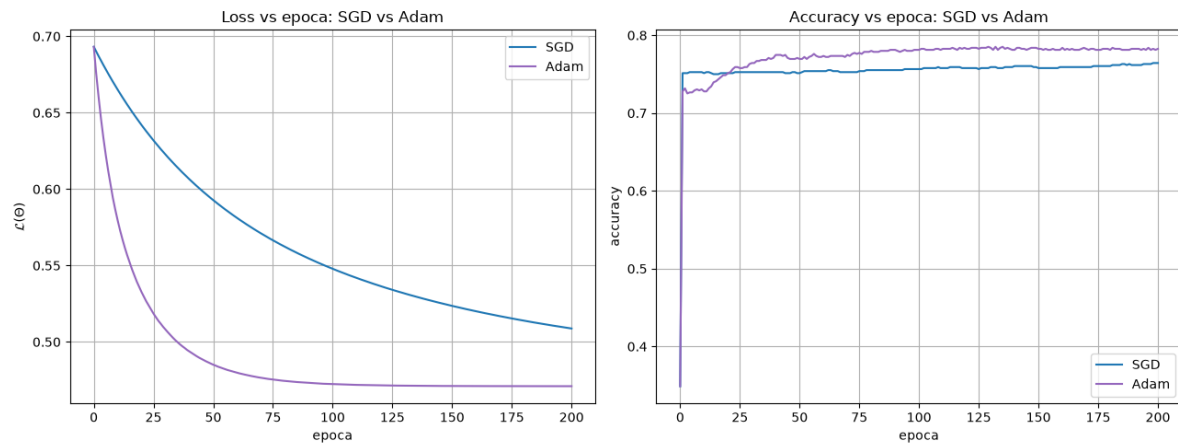
```
1 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
2 axes[0].plot(losses_sgd, color='tab:blue', label='SGD')
3 axes[0].plot(losses_adam, color='tab:purple', label='Adam')
4 axes[0].set_xlabel('epoca'); axes[0].set_ylabel(r'$\mathcal{L}(\backslash$
5   Theta)$')
6 axes[0].set_title('Loss vs epoca: SGD vs Adam')
7 axes[0].legend()
8
9 axes[1].plot(accs_sgd, color='tab:blue', label='SGD')
10 axes[1].plot(accs_adam, color='tab:purple', label='Adam')
11 axes[1].set_xlabel('epoca'); axes[1].set_ylabel('accuracy')
12 axes[1].set_title('Accuracy vs epoca: SGD vs Adam')
```

```

12 axes[1].legend()
13
14 plt.tight_layout()
15 plt.show()

```

Out [7]:



Loss (sinistra) e accuracy (destra) vs epoca, SGD (blu) vs Adam (viola), stessi iperparametri nominali.

In [8]:

```

1  # "Piu' rumoroso" a occhio e' facile da sbagliare, soprattutto se
    le due curve
2  # stanno su livelli diversi. Misuro la ruvidita' come media del
    valore assoluto
3  # della differenza seconda della curva (una retta, anche inclinata,
    ha differenza
4  # seconda nulla: resta solo l'oscillazione), sulle ultime 100
    epoche. Riporto
5  # anche lo spostamento medio dei parametri per epoca, che spiega il
    risultato.
6  def ruvidita(curva, ultime=100):
7      c = np.asarray(curva)[-ultime:]
8      return np.mean(np.abs(np.diff(c, n=2)))
9
10 print(f"{'metodo':>8} {'ruvidita loss':>16} {'ruvidita accuracy
    ':>20} ")
11     f"{'||dtheta|| medio/epoca':>24}")
12 for nome, ls, ac, ths in [('SGD', losses_sgd, accs_sgd, thetas_sgd)
    ,
13                             ('Adam', losses_adam, accs_adam,
                                thetas_adam)]:
14     step = np.mean(np.linalg.norm(np.diff(ths, axis=0), axis=1))
15     print(f"{'nome':>8} {'ruvidita(ls):>16.3e} {'ruvidita(ac):>20.3e} {
        step:>24.3e}")

```

Out [8]:

	metodo	ruvidita loss	ruvidita accuracy	dtheta medio/epoca
1	SGD	3.935e-06	5.713e-04	4.392e-03
2	Adam	8.160e-06	1.820e-03	1.106e-02

Un risultato che contraddice la narrativa standard “Adam è più liscio” — e vale la pena saperlo spiegare bene. Misurando la ruvidità in modo quantitativo (differenza seconda media sulle ultime 100 epoche, dove una retta anche inclinata darebbe zero: resta solo l’oscillazione vera), **Adam risulta più ruvido di SGD**, non più liscio: ruvidità della loss 8.16×10^{-6} contro 3.94×10^{-6} di SGD (più del doppio), stessa storia per l’accuracy (1.82×10^{-3} contro 5.71×10^{-4} , oltre 3 volte).

La spiegazione sta nell’ultima colonna: lo spostamento medio dei parametri per epoca è 1.106×10^{-2} per Adam contro 4.392×10^{-3} per SGD — Adam si muove $2.5 \times$ più velocemente ad ogni epoca. Questo non è un caso: l’update di Adam è $\Theta_{k+1} = \Theta_k - \eta \hat{m}_k / (\sqrt{\hat{v}_k} + \epsilon)$, e per parametri con gradiente storicamente piccolo (come accade qui, dato che $\kappa \approx 5.18$ è già ben condizionato dopo la standardizzazione), $\sqrt{\hat{v}_k}$ è piccolo, il che rende il passo effettivo $\eta / \sqrt{\hat{v}_k}$ molto più grande dello stesso η nominale usato da SGD. Muoversi più velocemente comporta oscillazioni proporzionalmente più ampie ad ogni passo, anche se il progresso complessivo (guardando la loss finale, 0.471 contro 0.509) è migliore.

In [9]:

```
1 metrics_sgd = compute_metrics(theta_sgd, X4, y4, threshold=0.5)
2 metrics_adam = compute_metrics(theta_adam, X4, y4, threshold=0.5)
3
4 for name, m in [('SGD', metrics_sgd), ('Adam', metrics_adam)]:
5     print(f"--- {name} ---")
6     print(f" Matrice di confusione: TP={m['TP']}, FP={m['FP']}, FN
7           = {m['FN']}, TN={m['TN']}")
8     print(f" Accuracy: {m['accuracy']:.4f}")
9     print(f" Precision: {m['precision']:.4f}")
10    print(f" Recall: {m['recall']:.4f}")
11    print(f" F1-score: {m['f1']:.4f}")
```

Out [9]:

```
1 --- SGD ---
2 Matrice di confusione: TP=159, FP=72, FN=109, TN=428
3 Accuracy: 0.7643
4 Precision: 0.6883
5 Recall: 0.5933
6 F1-score: 0.6373
7 --- Adam ---
8 Matrice di confusione: TP=157, FP=56, FN=111, TN=444
9 Accuracy: 0.7826
10 Precision: 0.7371
11 Recall: 0.5858
12 F1-score: 0.6528
```

Adam ha accuracy, precision e F1 leggermente migliori di SGD; il recall è quasi identico (0.586 vs 0.593). Coerente con la loss finale più bassa di Adam (0.471 contro 0.509): un modello leggermente meglio ottimizzato, sullo stesso budget di epoche.

In [10]:

```
1 p4 = sigmoid(X4 @ theta_adam)
2 print(f"punti con probabilit  predetta in [0.3, 0.7]: {np.sum((p4
3     > 0.3) & (p4 < 0.7))} su {N4}")
4 print()
5 print(f"{'soglia':10s} {'TP':>4s} {'FP':>4s} {'FN':>4s} {'TN':>4s}")
```

```

    {'accuracy':>9s} "
5     f"{'precision':>10s} {'recall':>8s} {'f1':>7s}")
6 for t in [0.3, 0.5, 0.7]:
7     m = compute_metrics(theta_adam, X4, y4, threshold=t)
8     print(f"{t:<10.1f} {m['TP']:>4d} {m['FP']:>4d} {m['FN']:>4d} {m['TN']:>4d} "
9           f"{m['accuracy']:>9.4f} {m['precision']:>10.4f} {m['recall']:>8.4f} {m['f1']:>7.4f}")

```

Out[10]:

```

1 punti con probabilita' predetta in [0.3, 0.7]: 234 su 768
2
3 soglia      TP    FP    FN    TN  accuracy  precision  recall    f1
4 0.3          214  142   54  358    0.7448    0.6011    0.7985  0.6859
5 0.5          157   56  111  444    0.7826    0.7371    0.5858  0.6528
6 0.7          100   22  168  478    0.7526    0.8197    0.3731  0.5128

```

Collegamento con l'Esercizio 3. Qui, a differenza del dataset giocattolo (dove solo 8 punti su 400 avevano probabilità non estrema), 234 punti su 768 ($\approx 30\%$) cadono in $[0.3, 0.7]$: è il caso in cui la soglia conta davvero. Il trade-off è netto: a soglia 0.3 il recall sale a 0.799 (pochi malati non diagnosticati, $FN = 54$) ma la precision crolla a 0.601 ($FP = 142$); a soglia 0.7 è l'opposto (precision 0.820, recall 0.373, $FN = 168$). In un contesto di screening medico come questo, dove mancare un caso di diabete (FN) è tipicamente più grave di un approfondimento diagnostico inutile (FP), la soglia 0.3 — nonostante l'accuracy complessiva più bassa (0.745 contro 0.783 a soglia 0.5) — sarebbe la scelta più sensata: è l'esempio concreto di perché l'accuracy da sola non basta a scegliere un modello.

Sintesi per la consegna: quale metodo converge più velocemente, quale oscilla di più, e il collegamento con i learning rate adattivi. Adam converge più velocemente in termini di loss (raggiunge in ~ 50 epoche un valore che SGD non raggiunge nemmeno dopo 200, si veda la Figura), ma è **Adam** a oscillare di più in valore assoluto (ruvidità doppia/tripla rispetto a SGD, tabella sopra) — non il contrario, come si potrebbe supporre a priori. La ragione è precisamente il meccanismo dei learning rate adattivi discusso a lezione: dividendo per $\sqrt{\hat{v}_k}$, Adam amplifica automaticamente il passo effettivo nelle direzioni dove il gradiente è stato storicamente piccolo (qui, un problema già ben condizionato dopo la standardizzazione, $\kappa \approx 5.18$) — il che accelera la convergenza iniziale ma produce anche spostamenti più ampi, e quindi oscillazioni più marcate, ad ogni singolo update rispetto a un passo fisso η come quello di SGD. Il vantaggio di Adam non è “essere più liscio”: è convergere più velocemente verso una loss più bassa, al prezzo di una dinamica locale meno regolare.

6 Estensione opzionale: da regressione logistica a una rete neurale

Consegna

Implement a neural network with one hidden layer, ReLU activation, sigmoid output layer:
 $h = \text{ReLU}(W_1x + b_1)$, $\hat{y} = \sigma(w_2^T h + b_2)$.

1. Train the neural network on the same Kaggle dataset.
2. Track BCE loss vs epoch, accuracy vs epoch.
3. Compare and discuss the performance against logistic regression.

Nota: this extension is not mandatory for passing the exam.

```
In [1]:

1  def relu(z):
2      return np.maximum(0, z)
3
4  def relu_deriv(z):
5      return (z > 0).astype(float)
6
7  class SimpleNN:
8      def __init__(self, n_in, n_hidden, seed=0):
9          rng_ = np.random.default_rng(seed)
10         # init casuale piccola, scalata di 1/sqrt(n_in)
11         self.W1 = rng_.normal(0, 1.0/np.sqrt(n_in), size=(n_in,
12             n_hidden))
13         self.b1 = np.zeros(n_hidden)
14         self.w2 = rng_.normal(0, 1.0/np.sqrt(n_hidden), size=
15             n_hidden)
16         self.b2 = 0.0
17
18     def forward(self, X_):
19         z1 = X_ @ self.W1 + self.b1
20         h = relu(z1)
21         z2 = h @ self.w2 + self.b2
22         yhat = sigmoid(z2)
23         cache = (X_, z1, h, z2, yhat)
24         return yhat, cache
25
26     def loss(self, X_, y_):
27         yhat, _ = self.forward(X_)
28         eps = 1e-12
29         return -np.mean(y_*np.log(yhat+eps) + (1-y_)*np.log(1-yhat+
30             eps))
31
32     def accuracy(self, X_, y_, threshold=0.5):
33         yhat, _ = self.forward(X_)
34         pred = (yhat >= threshold).astype(float)
35         return np.mean(pred == y_)
36
37     def backward(self, cache, y_):
38         X_, z1, h, z2, yhat = cache
39         n = X_.shape[0]
```

```

37         dz2 = (yhat - y_) / n
38         dw2 = h.T @ dz2
39         db2 = np.sum(dz2)
40         dh = np.outer(dz2, self.w2)
41         dz1 = dh * relu_deriv(z1)
42         dW1 = X_.T @ dz1
43         db1 = np.sum(dz1, axis=0)
44         return dW1, db1, dw2, db2
45
46     def step_adam(self, grads, m, v, t, eta, beta1=0.9, beta2
47 =0.999, epsilon=1e-8):
48         params = [self.W1, self.b1, self.w2, self.b2]
49         new_params = []
50         for i, (p, g) in enumerate(zip(params, grads)):
51             m[i] = beta1*m[i] + (1-beta1)*g
52             v[i] = beta2*v[i] + (1-beta2)*(g**2)
53             m_hat = m[i] / (1 - beta1**t)
54             v_hat = v[i] / (1 - beta2**t)
55             new_params.append(p - eta * m_hat / (np.sqrt(v_hat) +
56 epsilon))
57         self.W1, self.b1, self.w2, self.b2 = new_params
58         return m, v

```

Rete con un solo strato nascosto: $h = \text{ReLU}(W_1x + b_1)$, $\hat{y} = \sigma(w_2^T h + b_2)$. **backward** implementa la backpropagation applicando la chain rule a ritroso: $\partial\ell/\partial z_2 = \hat{y} - y$ (stesso risultato notevole della regressione logistica, Esercizio 1), poi $\partial\ell/\partial z_1 = (\hat{y} - y)w_2 \odot \text{ReLU}'(z_1)$, dove $\text{ReLU}'(z) = \mathbb{I}[z > 0]$ propaga il gradiente solo attraverso le unità “attive”. **step_adam** è lo stesso ottimizzatore usato per la regressione logistica nell’Esercizio 4, applicato qui a tutti e quattro i gruppi di parametri (W_1, b_1, w_2, b_2) separatamente.

Nota di implementazione: **X4** contiene già una colonna di bias (usata dalla regressione logistica), e la rete ha un proprio **b1**; è una ridondanza innocua, quella colonna si limita ad agire come un input costante in più per lo strato nascosto.

In [2]:

```

1  def train_nn_adam(X_, y_, n_hidden, eta, batch_size, n_epochs, seed
2  =0):
3      rng_ = np.random.default_rng(seed)
4      net = SimpleNN(X_.shape[1], n_hidden, seed=seed)
5      N_ = X_.shape[0]
6      m = [np.zeros_like(net.W1), np.zeros_like(net.b1), np.
7           zeros_like(net.w2),
8           np.zeros_like(np.array(net.b2))]
9      v = [np.zeros_like(net.W1), np.zeros_like(net.b1), np.
10           zeros_like(net.w2),
11           np.zeros_like(np.array(net.b2))]
12      t = 0
13      losses = [net.loss(X_, y_)]
14      accs = [net.accuracy(X_, y_)]
15      for epoch in range(n_epochs):
16         idx = rng_.permutation(N_)
17         for start in range(0, N_, batch_size):
18             b_idx = idx[start:start+batch_size]
19             Xb, yb = X_[b_idx], y_[b_idx]
20             yhat, cache = net.forward(Xb)

```

```

18         grads = net.backward(cache, yb)
19         t += 1
20         m, v = net.step_adam(grads, m, v, t, eta)
21         losses.append(net.loss(X_, y_))
22         accs.append(net.accuracy(X_, y_))
23     return net, np.array(losses), np.array(accs)
24
25 net_nn, losses_nn, accs_nn = train_nn_adam(X4, y4, n_hidden=8, eta
    =1e-3,
26                                     batch_size=32, n_epochs
    =200, seed=4)
27 print(f"Rete neurale (Adam): loss finale = {losses_nn[-1]:.4f}, "
28       f"accuracy finale = {accs_nn[-1]:.4f}")
29 print(f"numero di parametri: rete = "
30       f"{net_nn.W1.size + net_nn.b1.size + net_nn.w2.size + 1}, "
31       f"regressione logistica = {d4}")

```

Out[2]:

```

1 Rete neurale (Adam): loss finale = 0.4290, accuracy finale = 0.7917
2 numero di parametri: rete = 89, regressione logistica = 9

```

Stessi iperparametri di ottimizzazione della regressione logistica (Adam, $\eta = 10^{-3}$, batch 32, 200 epoche), 8 unità nascoste. Da notare subito: la rete ha 89 parametri contro i 9 della regressione logistica — $\approx 10\times$ più capacità, un fatto che va tenuto presente in tutto il confronto che segue.

In [3]:

```

1 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
2 axes[0].plot(losses_adam, color='tab:purple', label='Regressione
    logistica (Adam)')
3 axes[0].plot(losses_nn, color='tab:green', label='Rete neurale (
    Adam)')
4 axes[0].set_xlabel('epoca'); axes[0].set_ylabel(r'$\mathcal{L}$')
5 axes[0].set_title('Loss vs epoca')
6 axes[0].legend()
7
8 axes[1].plot(accs_adam, color='tab:purple', label='Regressione
    logistica (Adam)')
9 axes[1].plot(accs_nn, color='tab:green', label='Rete neurale (
    Adam)')
10 axes[1].set_xlabel('epoca'); axes[1].set_ylabel('accuracy')
11 axes[1].set_title('Accuracy vs epoca')
12 axes[1].legend()
13
14 plt.tight_layout()
15 plt.show()
16
17 yhat_nn, _ = net_nn.forward(X4)
18 pred_nn = (yhat_nn >= 0.5).astype(float)
19 TP, FP, FN, TN = confusion_matrix_counts(y4, pred_nn)
20 precision_nn = TP/(TP+FP) if (TP+FP)>0 else 0.0
21 recall_nn = TP/(TP+FN) if (TP+FN)>0 else 0.0
22 f1_nn = 2*precision_nn*recall_nn/(precision_nn+recall_nn) if (
    precision_nn+recall_nn)>0 else 0.0

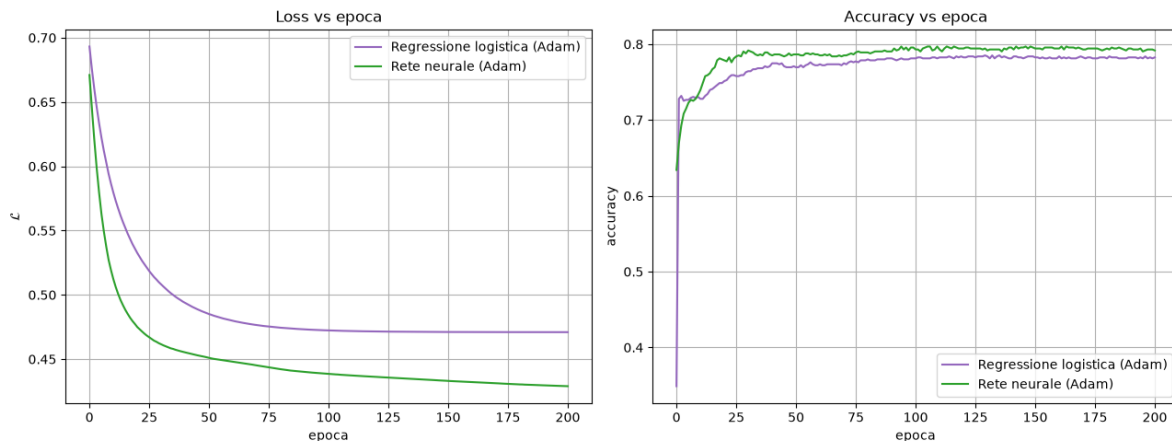
```

```

23 print(f"Rete neurale - matrice di confusione: TP={TP}, FP={FP}, FN
    ={FN}, TN={TN}")
24 print(f"Rete neurale - precision={precision_nn:.4f}, recall={
    recall_nn:.4f}, F1={f1_nn:.4f}")

```

Out [3]:



Loss (sinistra) e accuracy (destra) vs epoca: regressione logistica (viola) vs rete neurale (verde), stessi iperparametri di ottimizzazione.

```

1 Rete neurale - matrice di confusione: TP=168, FP=60, FN=100, TN=440
2 Rete neurale - precision=0.7368, recall=0.6269, F1=0.6774

```

Confronto diretto (a soglia 0.5, tutte le metriche sul training set).

	loss finale	accuracy	precision	recall (F1)
Regressione logistica (Adam)	0.471	0.783	0.737	0.586 (0.653)
Rete neurale (Adam)	0.429	0.792	0.737	0.627 (0.677)

La rete raggiunge una loss finale più bassa (0.429 contro 0.471) e supera la regressione logistica su quasi tutte le metriche (accuracy, recall, F1), a parità di precision. Guardando le curve: nel pannello di sinistra la loss della rete (verde) parte più in alto ma scende più rapidamente e si stabilizza sistematicamente *sotto* quella della logistica (viola) per tutta la seconda metà delle epoche — un divario che si allarga leggermente nel tempo, non si richiude. Nel pannello destro le due curve di accuracy si incrociano più volte nelle prime 50 epoche, poi la rete si assesta stabilmente un po' sopra.

Perché la rete può fare meglio: la capacità del modello. La regressione logistica produce sempre un confine di decisione lineare (Esercizio 1): può solo separare le classi con un iperpiano nello spazio delle 8 feature originali. Lo strato nascosto ReLU introduce non linearità: la rete può comporre più iperpiani (uno per ciascuna delle 8 unità nascoste) in un confine di decisione complessivamente non lineare — maggiore *capacità* di adattarsi a strutture più complesse nei dati, come sembra essere il caso qui.

L'avvertenza necessaria. Questo confronto è interamente sul **training set**, e la rete ha 89 parametri contro i 9 della regressione logistica, quasi 10× più capacità. Un divario di loss crescente nel tempo tra un modello con più parametri e uno con meno, misurato solo sui dati di addestramento, è esattamente il pattern che ci si aspetterebbe anche in presenza di **overfitting**: la rete potrebbe semplicemente star memorizzando dettagli specifici del

training set piuttosto che imparare una relazione che generalizza meglio. Per sapere se il miglioramento è reale servirebbe valutare entrambi i modelli su un validation/test set separato, mai visto durante l'addestramento — cosa che va oltre lo scopo di questa estensione opzionale, ma che è il passo naturale successivo per una conclusione solida sul confronto capacità/generalizzazione.